

SISTEMAS DISTRIBUIDOS I
(TA050) CURSO PABLO D. ROCA

Trabajo Práctico Diseño Coffee Shop Analysis

× ×

16 de septiembre de 2025

Sebastian Mauricio Vintoñuke	106063
Marcelo Agustin Origoni	109903
Agustin Pelliciari	108172

1. Alcance

El sistema tiene como propósito procesar y analizar información de ventas en una cadena de cafés en Malasia.

A través de un diseño distribuido, busca obtener métricas a partir de datos transaccionales, de clientes, tiendas y productos.

Alcance funcional, permite obtener:

- Transacciones realizadas entre 2024 y 2025 entre las 06:00 AM y las 11:00 PM con monto total mayor a 15.
- Nombre del producto más vendido y nombre del producto que más ganancias ha generado, para cada mes en 2024 y 2025.
- TPV (Total Payment Value) por cada semestre en 2024 y 2025, para cada sucursal, para transacciones realizadas entre las 06:00 AM y las 11:00 PM.
- Fecha de cumpleaños de los 3 clientes que han hecho más compras entre 2024 y 2025, para cada sucursal.

Alcance no funcional:

- Optimizado para entornos multicomputadoras.
- Soporta el incremento de los elementos de cómputo para escalar los volúmenes de información a procesar.
- Usa un Middleware para abstraer la comunicación basada en grupos.
- Soportar una única ejecución del procesamiento y provee graceful quit frente a señales SIG-TERM.

2. Arquitectura de Software

El sistema se basa en una arquitectura mixta. Adopta un modelo cliente-servidor para gestionar la interacción con los usuarios. Internamente, está conformado por múltiples componentes de cómputo que trabajan en conjunto, lo que permite atender los requerimientos de escalabilidad y aprovechar las ventajas de los entornos multicómputo.

2.1. Interacción con el cliente

La comunicación entre el cliente y el sistema se basa en un modelo cliente-servidor. El cliente envía una consulta en la que incluye los archivos CSV a procesar. El servidor recibe esta petición y responde con un `requestId`, un identificador único asociado a la consulta.

Más adelante, el cliente utiliza ese `requestId` para consultar el estado de su solicitud. Esta operación es bloqueante, ya que el cliente debe esperar hasta que el procesamiento haya finalizado para poder acceder a los resultados.

En conjunto, la interacción define un **protocolo de comunicación asíncrono**, estructurado en dos intercambios de tipo request-reply sincrónicos: el primero para registrar la consulta y obtener el identificador, y el segundo para recuperar los resultados una vez estén disponibles.

2.2. Interacción entre nodos

La interacción principal del sistema se establece entre el Dispatcher y los nodos de ejecución, responsables de llevar a cabo las distintas etapas de procesamiento que conforman los pipelines.

Esta coordinación se realiza a través de un **Message-Oriented Middleware (MOM)**, centralizado y basado en un modelo de colas asíncronas de mensajes. En este esquema, cada nodo tiene una cola de entrada y despacha sus resultados a una siguiente cola, exceptuando el último nodo, el cual consolida los resultados y los envía al almacenamiento persistente de resultados.

3. Vista Lógica

3.1. Entidades del dominio del negocio

Menu Item: Representa los productos del menú disponibles en cada tienda.

Store: Sucursales donde se realizan las transacciones.

Transaction Item: Relación entre un producto y una transacción específica, con cantidad y precio unitario.

Transaction: Operación de compra realizada por un cliente en una tienda en una fecha y hora determinada.

User: Cliente que interactúa con la cadena de cafés.

3.2. Entidades del dominio del sistema

Servidor: Punto de entrada al sistema. Gestiona las solicitudes de los clientes, retorna los resultados y coordina la comunicación con el Dispatcher.

Dispatcher: Responsable de recibir los archivos, validar y planificar la ejecución de las consultas. Divide el trabajo en tareas y las envía a los distintos nodos de procesamiento.

Middleware (MOM): Componente centralizado de mensajería orientada a colas. Permite la comunicación asincrónica entre nodos y asegura la tolerancia a fallos.

Select Node: Nodo encargado de aplicar filtros (por fecha, hora y monto) para reducir el volumen de datos antes de etapas más complejas.

Group Node: Nodo que agrupa la información según distintos criterios (por ejemplo, por producto, por cliente o por sucursal) y calcula métricas agregadas.

Join Node: Nodo que une los resultados parciales con otras entidades del dominio (ejemplo: asociar item_id con Menu Item).

Result Node: Consolida los resultados de las consultas, los persiste en almacenamiento y los deja listos para ser recuperados por el cliente.

State Storage: Almacenamiento intermedio utilizado por nodos que requieren mantener estado, como los Group o Join Nodes.

Result Storage: Repositorio persistente donde se guardan los resultados finales de cada consulta, asociados al requestId y al cliente.

Task Queue: Cola de mensajes que intermedia entre los distintos nodos y asegura la distribución ordenada de las tareas de procesamiento.

4. Vista de Procesos

La planificación de las consultas se diseñó con el objetivo de reducir el volumen de datos lo antes posible. Para ello, se aplican en primer lugar las proyecciones, eliminando las columnas irrelevantes, y los selects, descartando las filas que no cumplen los criterios de filtrado. Este enfoque no solo optimiza la ejecución al minimizar la cantidad de datos procesados en las etapas posteriores, sino que además reduce la complejidad de operaciones más costosas como group by y join.

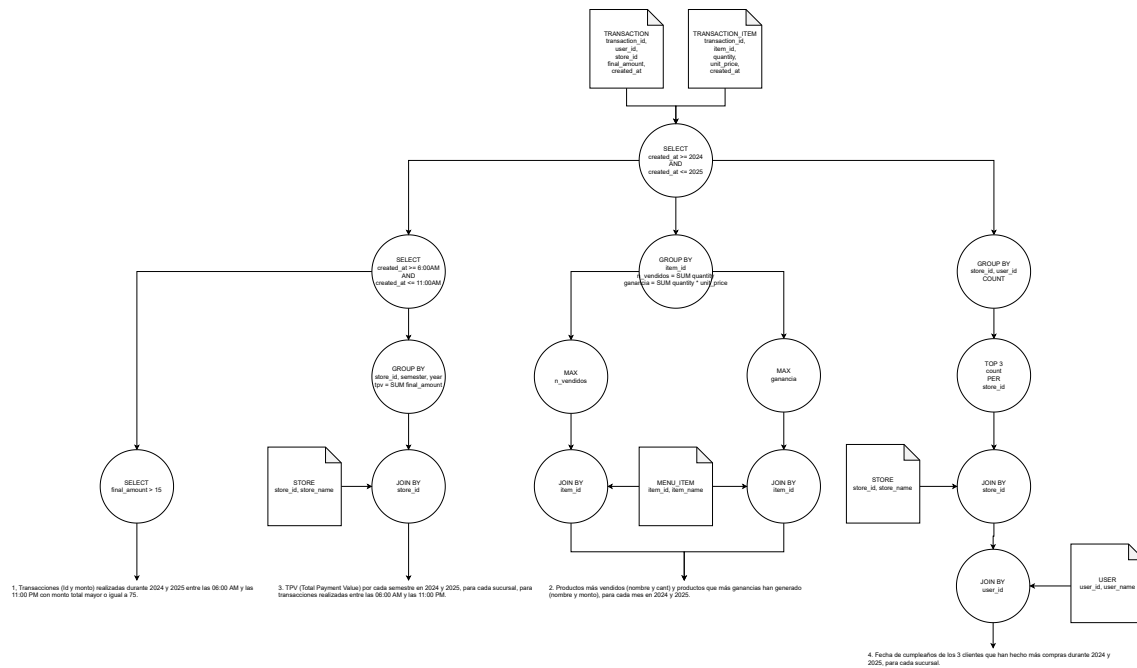


Figura 1: Diagrama DAG: Planificación de consultas

Ciertas tareas se pueden realizar en paralelo, por ejemplo la selección puede dividirse entre varios nodos ya que no es una operación que requiere de la totalidad de los datos para resolverse, a continuación se muestra como dos nodos de selección trabajan en paralelo.

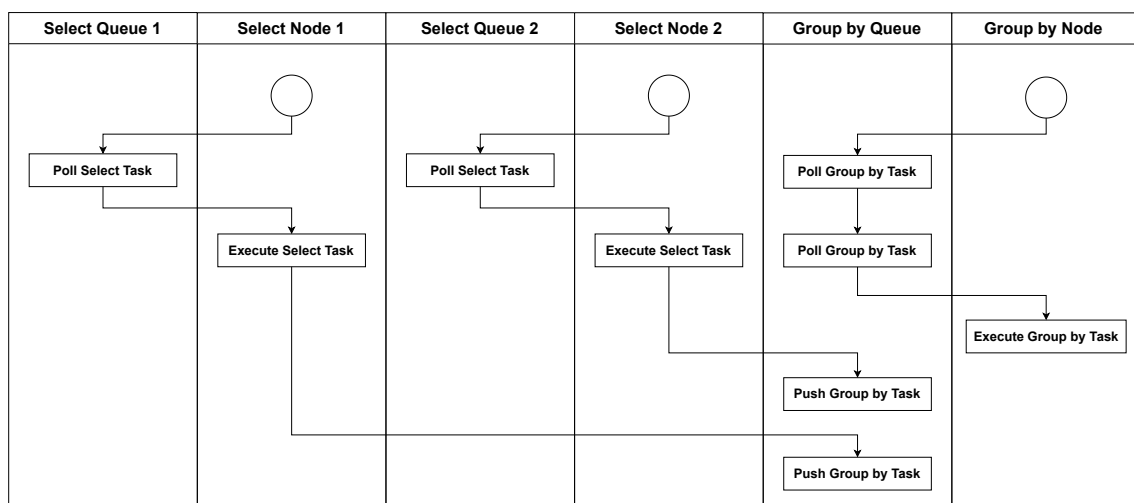


Figura 2: Diagrama de actividades: Selección paralela

Este diagrama describe cómo un Select Node procesa las tareas que recibe a través del middleware.

El SelectNode consume las tareas desde la cola correspondiente utilizando `consume_select_tasks`. Cada tarea es derivada a un TypeHandler, que se obtiene con `get_handler(task.type_query)`. El handler ejecuta la operación (`exec(task.data)`), produciendo un resultado parcial (`task_output`). El resultado se envía al Middleware mediante `send_output`, que utiliza un esquema de hashing (`hash(key)`) para determinar a qué GroupQueue debe ir. Finalmente, el GroupNode correspondiente recibe la tarea y ejecuta `handle_group(task)`, continuando con la siguiente etapa del pipeline.

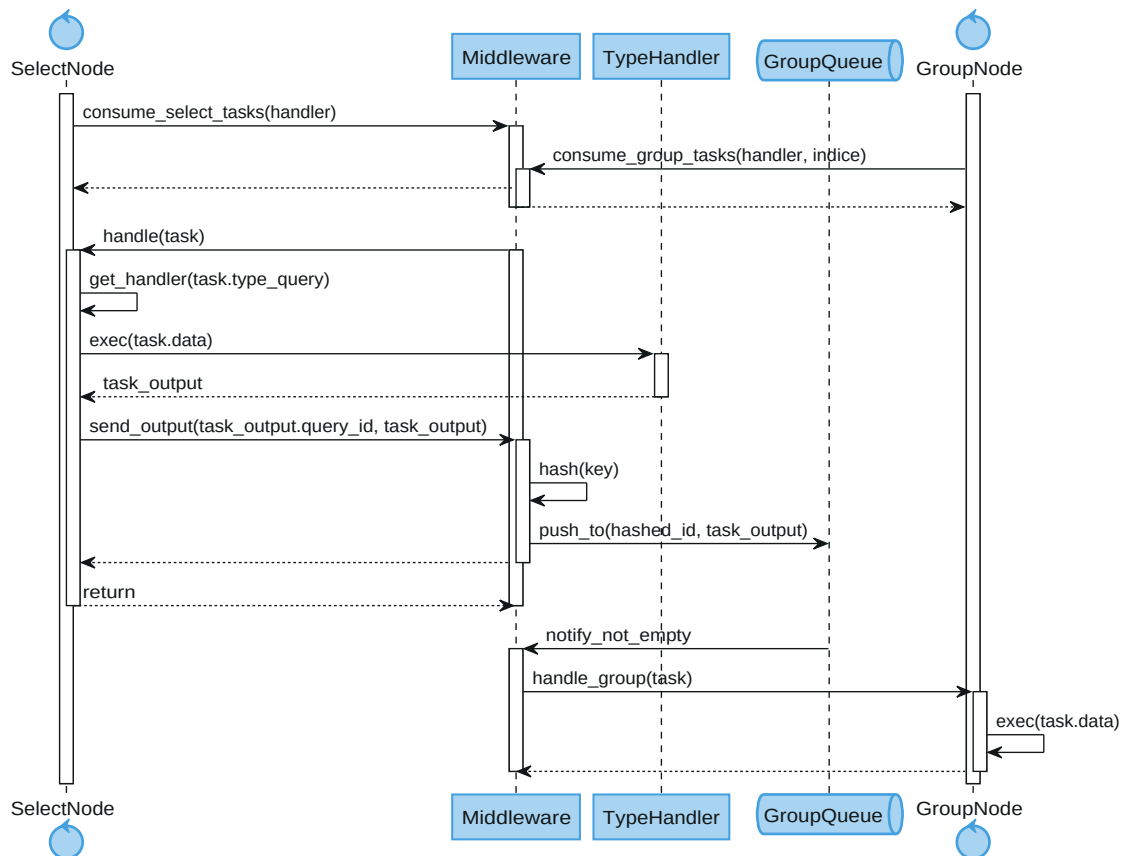


Figura 3: Diagrama de secuencia: Select Node

Este diagrama describe cómo un Group Node procesa las tareas que recibe a través del middleware.

El GroupNode recibe tareas desde el middleware (`handle(task)`). Se obtiene el handler correspondiente mediante `get_handler(task.type_query)`. El handler ejecuta la lógica de agrupación (`exec(task.data)`), generando resultados junto con una clave de partición (`task_output, hash_key`). El resultado se envía al middleware con `send_output(hash_key, task_output)`, que utiliza `hash(hash_key)` para decidir a qué JoinQueue corresponde. El JoinNode recibe los resultados parciales y continúa con las uniones necesarias.

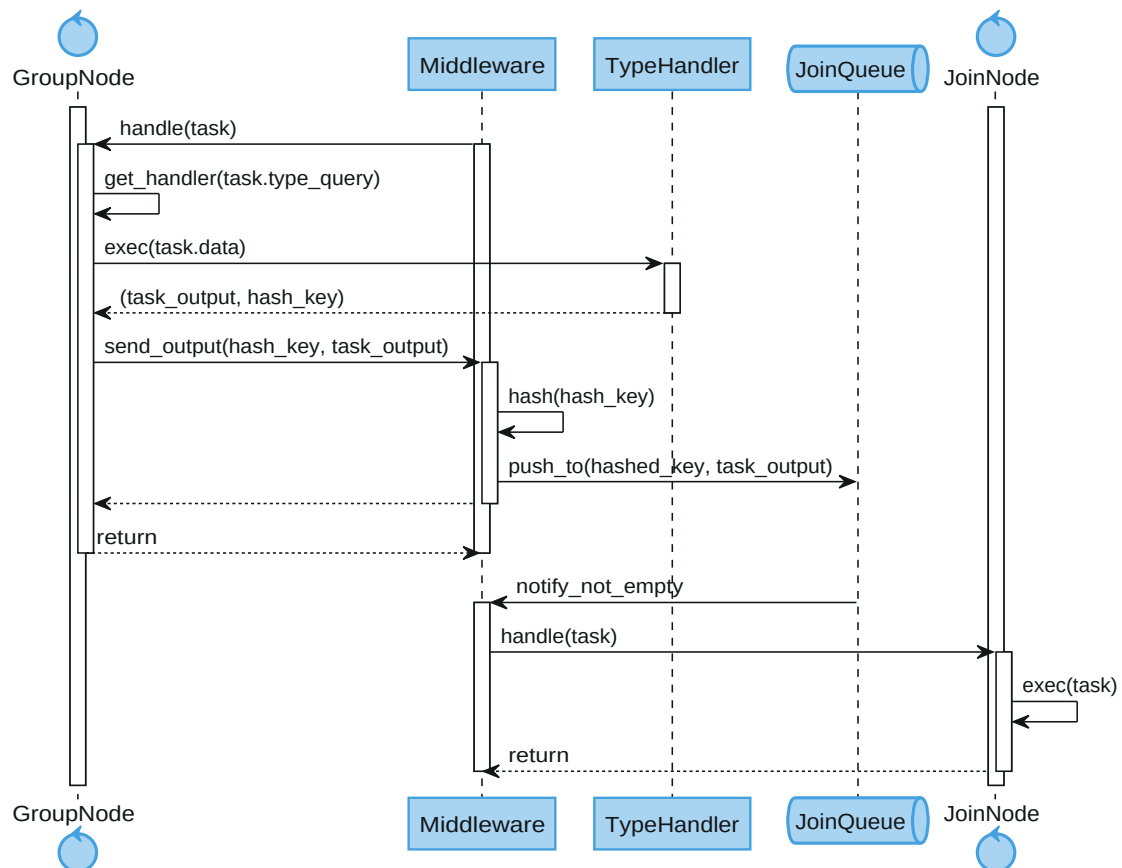


Figura 4: Diagrama de secuencia: Group Node

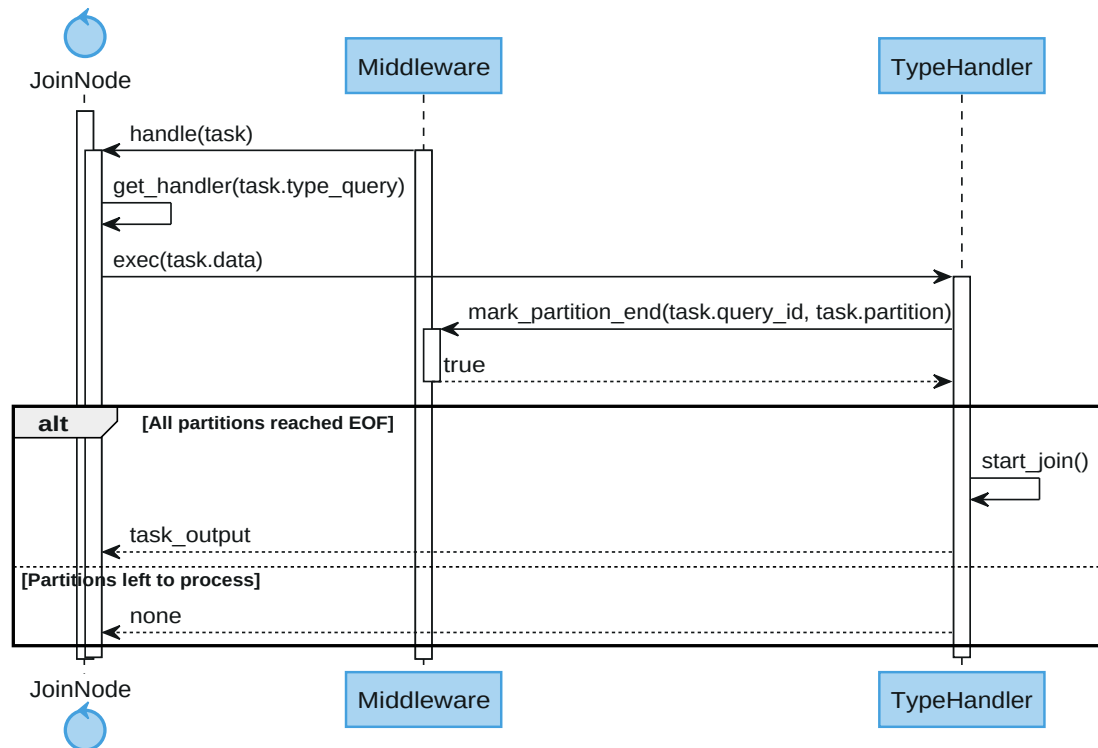


Figura 5: Diagrama de secuencia: Particion de un archivo

5. Vista de Desarrollo

Business Logic: Contiene la lógica de negocio asociada a las consultas definidas en el alcance. Aquí se implementan las reglas para filtrar, agrupar, unir y consolidar datos, de manera independiente de la infraestructura subyacente.

Processing Nodes: Incluye las implementaciones de los distintos nodos de procesamiento (Select, Group, Join, Result). Cada nodo ejecuta tareas específicas del pipeline y se comunica con otros a través del middleware.

Type Handlers: Define cómo se va a comportar un nodo en función de la consulta que se está ejecutando en él.

Middleware: Proporciona la abstracción de comunicación entre nodos mediante un modelo orientado a colas. Se asegura de la entrega confiable de mensajes, la asincronía y la tolerancia a fallos.

State Storage: Almacenamiento intermedio para nodos con estado (Group y Join). Garantiza que los datos parciales se conserven mientras se procesan.

Result Storage: Repositorio persistente para almacenar los resultados finales, asociados a cada consulta y cliente mediante el requestId.

Routing: Encargado de dirigir cada mensaje hacia el nodo o cola correspondiente, aplicando estrategias de hash y balanceo de carga.

Business Protocol: Define el contrato de comunicación entre cliente y servidor (registro de consultas, consulta de resultados, manejo de identificadores).

Task Protocol: Describe cómo se representan, envían y reciben las tareas de procesamiento entre nodos. Incluye metadatos como tipo de consulta, identificador de cliente y partición.

Data Protocol: Normaliza el formato de los datos transaccionales y de negocio (transacciones, usuarios, productos, sucursales) que circulan en el sistema.

Byte Protocol: Capa más baja de comunicación, que define la serialización y deserialización de los mensajes para su envío eficiente a través del middleware.

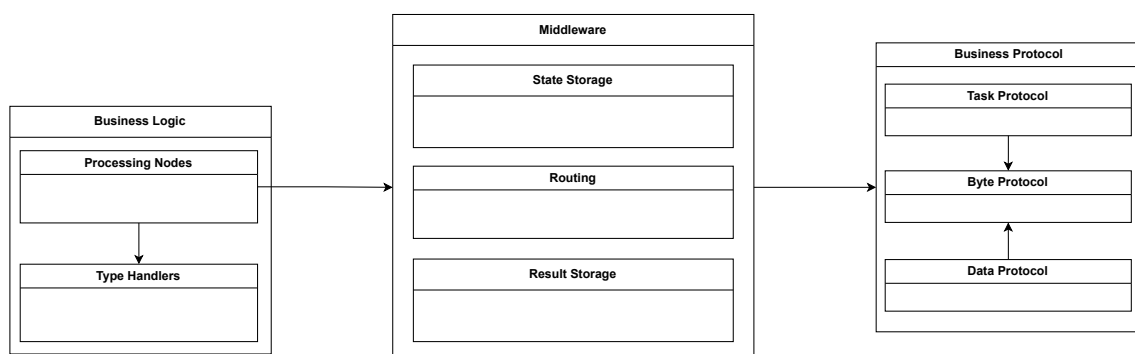


Figura 6: Diagrama de paquetes

6. Vista Física

6.1. Topología compartida

La **topología del sistema** se define a partir de los distintos pipelines necesarios para resolver cada consulta. Todas las consultas comparten una sección común formada por tres componentes principales.

Servidor: Actúa como balanceador de carga: cuando un usuario envía una consulta, lo conecta con un Dispatcher; en cambio, cuando el usuario solicita acceder a los resultados, el servidor se encarga de devolverlos una vez que la ejecución ha finalizado.

Resultados: Los resultados de cada consulta se almacenan en disco y se asocian al ID del cliente que la originó, lo que permite acceder a ellos de manera asincrónica en cualquier momento posterior.

Dispatcher: recibe los datos, aplica las validaciones necesarias y transforma su formato cuando es conveniente. Además, se encarga de planificar el pipeline de nodos y de reenviar los datos a través de este para su procesamiento.

6.2. Topología entre consultas

Para explicar la topología del sistema, la organizamos en función de los distintos pipelines asociados a cada consulta. Sin embargo, es importante destacar que como muchos de los datos iniciales y de los resultados parciales son compartidos por múltiples consultas. Para optimizar la resolución y evitar cálculos redundantes, los nodos pueden retransmitir sus resultados a varios pipelines de manera simultánea, lo que permite que diferentes consultas reutilicen la misma información.

6.3. Topología por consulta

En la consulta 1, cada **Select Node** es capaz de ejecutar las tres selecciones requeridas. Para acelerar la resolución, es posible utilizar múltiples nodos en paralelo. Los resultados generados se almacenan en un nodo de resultados **Result Node** y, una vez completada la obtención de todos los datos, se guardan en el almacenamiento persistente de resultados.

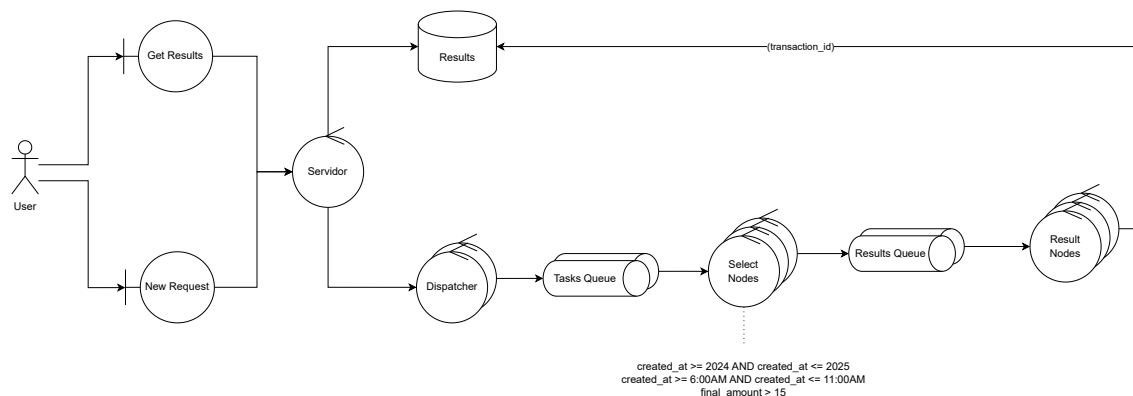


Figura 7: Diagrama de robustez consulta 1°

En la consulta 2, cada **Select Node** únicamente realiza la primera selección por año. Posteriormente, los resultados se dirigen a un **Group Node**, el cual calcula dos métricas: el producto más vendido y el de mayor ganancia. Estos datos se bifurcan en dos pipelines independientes, que realizan un join y consolidan la información en el **Result Node**. Una vez unificados, los datos se almacenan de forma persistente, del mismo modo que en la consulta 1.

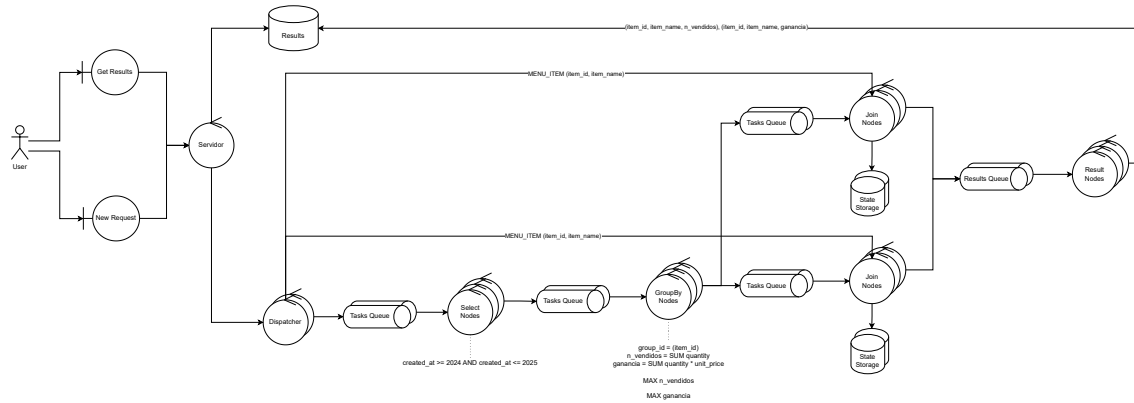


Figura 8: Diagrama de robustez consulta 2°

En la consulta 3 se reutiliza el resultado parcial de la consulta 1, correspondiente a la selección por año y hora. A continuación, el **Group Node** agrupa los datos y calcula el TPV. Finalmente, se realiza un join, tras el cual los resultados se consolidan y se almacenan en disco.

Es importante destacar que, como se mencionó previamente, las consultas 1 y 3 utilizan los mismos datos en su etapa inicial (la selección por año). Aunque se representen en gráficos separados, este cálculo se realiza una sola vez y se retransmite a múltiples pipelines.

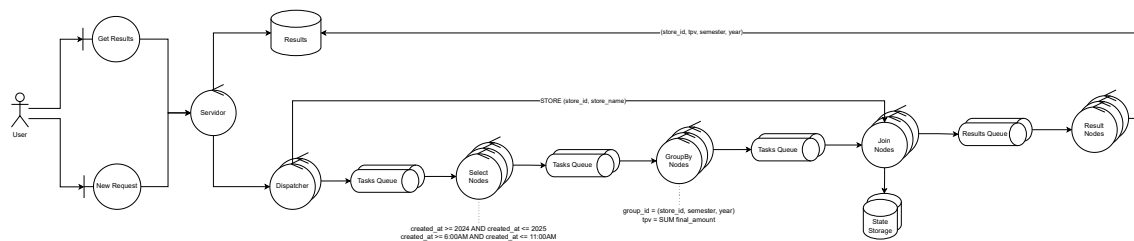


Figura 9: Diagrama de robustez consulta 3°

En la consulta 4 también se reutiliza el resultado parcial de la consulta 1, correspondiente a la selección por año. El **Group Node** ejecuta los cálculos requeridos por la consulta, en este caso, la obtención del top 3 de clientes con más compras. Finalmente, se realizan dos joins consecutivos y, al igual que en los casos anteriores, los datos se consolidan y se almacenan en disco.

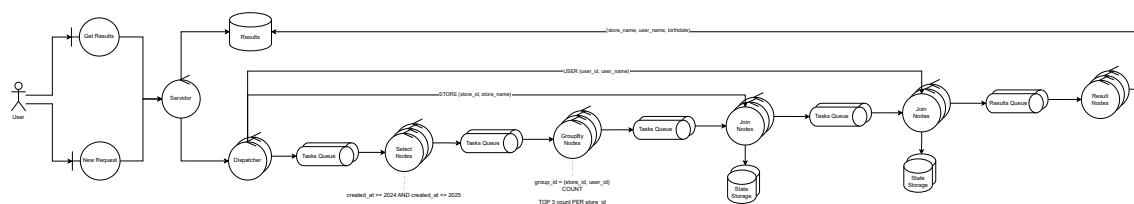


Figura 10: Diagrama de robustez consulta 4°

6.4. Despliegue

El **Servidor**, el **Middleware** y el almacenamiento de **Resultados** se despliegan como componentes independientes. Para aumentar la capacidad de procesamiento de solicitudes concurrentes, se permite el despliegue de múltiples instancias de **Dispatcher**. Las **Select Task Queues** y los **Select Nodes** también se despliegan de manera desacoplada, dado que cualquier nodo puede consumir tareas de cualquier cola. En contraste, los nodos que requieren mantener estado, como **Join** y **Group**, deben desplegarse junto con su cola y su almacenamiento asociados, a fin de preservar la coherencia del procesamiento. De manera análoga, los **Result Nodes** se despliegan con su cola correspondiente. En estos últimos dos casos, la asociación estricta con una cola garantiza la consistencia en la asignación de consultas a nodos específicos.

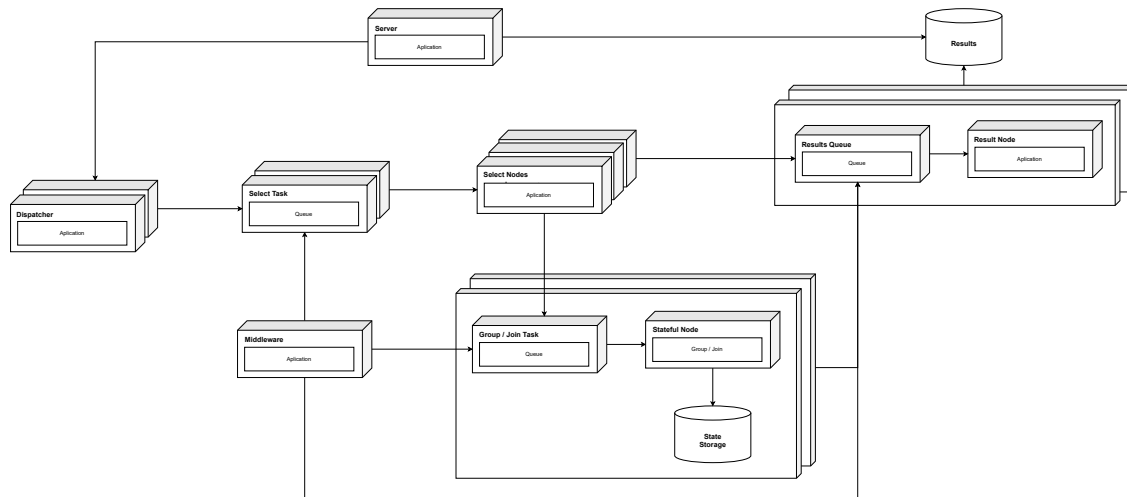


Figura 11: Diagrama de despliegue